

Technical Deep Dive

Phases 0 & 1, explained as computer science — every concept, every term, every step

24 July 2026 · Standalone reference document

Contents

1. Part I — System Architecture

2. 1. How the whole system is shaped, and why

3. Part II — Phase 0 Concepts

4. 2. Version control with Git

5. 3. Python virtual environments

6. 4. Dependency management and version resolution

7. 5. Python packages and modules

8. 6. Environment variables and configuration

9. 7. Node.js, npm, and package.json

10. 8. Vite as a build tool

11. 9. React and component-based UI

12. 10. Tailwind CSS v4's plugin architecture

13. 11. Progressive Web Apps

14. 12. The PNG file format, from first principles

15. 13. Client-server architecture and Backend-as-a-Service

16. 14. API keys and JSON Web Tokens

17. Part III — Phase 1 Concepts

18. 15. Relational database fundamentals

19. 16. SQL DDL — building the schema

20. 17. UUIDs as primary keys

21. 18. Foreign keys and referential integrity

22. 19. JSONB columns

23. 20. Indexes

24. 21. Row Level Security, in depth

25. 22. The Repository Pattern

26. 23. REST over Postgres — how the client actually talks to the database

27. 24. Testing: smoke tests and round-trip verification

28. 25. Networking — DNS, IPv4 vs IPv6, connection pooling

29. 26. Credential hygiene and the principle of least privilege

30. Part IV — Reference

31. 27. Every file created, by category

32. 28. Glossary of every term used

1. How the whole system is shaped, and why

Before the individual pieces, the overall shape. This project is two separate programs that share one database:

- An **agent** — a Python program with no user interface. It runs on a schedule, on a server, with nobody watching it. It browses websites, calls an AI model, and writes results into the database.
- A **dashboard** — a React web application. It has no logic of its own about leads or scoring; it only displays what is in the database and lets a human edit settings or approve messages.

This is a standard pattern called **separation of concerns**: each program has exactly one job, and the only thing connecting them is shared data, not shared code. The practical benefit is that either program can be stopped, redeployed, or rewritten without touching the other. If the dashboard crashes, the agent keeps running a scheduled job unaffected; if the agent needs a code fix, the dashboard stays live for Mahmoud to use.

Why not one program?

A single combined program (say, a Python web server that both browsed the internet and served the dashboard) would couple two very different workloads — long-running browser automation and low-latency user requests — onto one process. A crash or a slow browser session would take the dashboard down with it. Splitting them removes that coupling entirely.

2. Version control with Git

Git is a system for tracking changes to a set of files over time. Instead of saving "final_v2_FINAL.py" copies, every change is recorded as a **commit** — a snapshot of the entire project at one moment, with a message describing what changed and who changed it. Commits form a chain; each one points to the commit before it, so the whole history can be walked backwards.

Local versus remote

A Git **repository** (or "repo") starts out entirely on one machine — running `git init` created a hidden `.git` folder in the project directory that stores the entire history. This is the **local** repo. A **remote** is a copy of that same history hosted somewhere else — in this case, GitHub. The two are connected by a named pointer called `origin`, set once with `git remote add origin <url>`.

`git commit` only ever touches the local repo. `git push` is the separate act of uploading local commits to the remote so they exist on GitHub too. This project deliberately keeps those two actions performed by different people: every commit in this repository was made and pushed under Hussein's own Git identity, so that the project's authorship record on GitHub is accurate.

The staging area

Git has an intermediate step between "a file changed on disk" and "a file is part of the next commit," called the **staging area** (or "index"). Running `git add <file>` moves a change into staging; `git commit` then snapshots only what is staged. This exists so a commit can be built up deliberately — staging exactly the files that belong to one logical change — rather than always committing every modified file at once.

Why .gitignore was the very first file written

A `.gitignore` file tells Git which files to never track, by pattern. It was written before the very first `git add` in this project for one specific reason: this repository is **public** on GitHub. If a secret (an API key, a database password) is committed even once, it remains recoverable from the project's history forever, even after being deleted in a later commit — anyone can check out an old commit and read it, and automated bots actively scan public GitHub repositories for exactly this pattern. Writing the ignore rules first means a secret file physically cannot enter the staging area by accident.

3. Python virtual environments

Python packages are normally installed into one shared location per Python installation on a machine. This becomes a problem the moment two different projects need two different, and possibly incompatible, versions of the same package. A **virtual environment** (venv) solves this by creating an isolated, self-contained copy of the Python interpreter and its package directory, scoped to one project folder.

Running `python -m venv agent/venv` created `agent/venv/`, containing its own `python.exe` and its own empty package directory. From that point forward, invoking `agent/venv/Scripts/python.exe` instead of the system's `python` guarantees every package import resolves against this project's isolated set, never anything installed globally on the machine or in any other project's venv. This is why every command in this project's documentation calls that specific path rather than the bare word `python`.

`requirements.txt` is the declaration of what belongs in that isolated environment. Running `pip install -r requirements.txt` reads that file and installs exactly those packages (and their own dependencies) into the venv. Because the venv folder itself is never committed to Git (it is large, and entirely regenerable), any machine — a laptop, a server — can recreate an identical environment from just the requirements file.

4. Dependency management and version resolution

Each package in `requirements.txt` can declare which versions of the packages it depends on are acceptable — for example, the `anthropic` package might state it needs the `httpx` package at version 0.27 or higher. When two packages in the same environment place *conflicting* constraints on a shared dependency (one needs `httpx` below version 1, another needs it pinned to exactly 0.28), the installer has to find a single version that satisfies both, or fail.

What actually happened here: the file originally pinned every package to one *exact* version using `==`. That is the strictest possible constraint — it says "only this version, nothing else is acceptable" — and it left no room for `pip`'s solver to reconcile `supabase`'s and `anthropic`'s differing requirements on `httpx`. The fix was to loosen most pins to `>=` ("this version or newer") and remove the pin on `httpx` entirely, letting the solver pick whichever version satisfies every constraint simultaneously. This is a standard trade-off: exact pins give perfect reproducibility but zero flexibility; minimum-version pins give the solver room to work.

5. Python packages and modules

A folder containing an `__init__.py` file is treated by Python as a **package** — a namespace that can contain other files (**modules**) and be imported with dot notation, e.g. `from agent.db import repositories` reaches into the `agent` package, its `db` sub-package, and the `repositories` module inside it. Without an `__init__.py`, Python has no guarantee the folder is meant to be importable, and in some configurations the import simply fails. Nine empty `__init__.py` files were created for exactly this reason — one per folder (`agent`, `core`, `discovery`, `analysis`, `messaging`, `sending`, `crm`, `notifications`, `db`).

6. Environment variables and configuration

An **environment variable** is a named value that lives outside of any source code file, provided by the operating system or process environment at run time. They are the standard mechanism for supplying configuration — especially secrets — to a program without hard-coding them into files that get committed to version control.

This project uses a `.env` file (loaded by the `python-dotenv` library) to populate environment variables during local development. `config.py` is the only file that calls `os.getenv()` to read them; everywhere else in the codebase asks `config` for a value by name. This is a deliberate centralisation: if a variable is ever renamed, or a new one added, there is exactly one file to change, and every other module is unaffected because it never knew the environment variable's name in the first place — it only knows `config.SUPABASE_URL`.

7. Node.js, npm, and package.json

The dashboard is written in JavaScript, but JavaScript needs a runtime to execute outside a browser — that runtime is **Node.js**. **npm** (Node Package Manager) is both a public registry of reusable JavaScript packages and the CLI tool used to install them. `package.json` is JavaScript's equivalent of `requirements.txt`: it declares which packages the project depends on (`dependencies` , needed when the app actually runs — React, the Supabase client) and which are only needed while building it (`devDependencies` — Vite, Tailwind). Running `npm install` reads this file and downloads every listed package, plus their own dependencies, into a folder called `node_modules/` — the JavaScript equivalent of the Python `venv`, and likewise never committed to Git.

8. Vite as a build tool

Browsers cannot directly run the kind of code a modern React project is written in — it uses syntax (JSX, ES module imports of `.jsx` files) that no browser understands natively. A **build tool** transforms that source code into plain JavaScript, HTML, and CSS that a browser can execute. **Vite** is the build tool used here. During development, `npm run dev` starts a local server that transforms files on the fly as they are requested, which is why editing a component updates the browser almost instantly. For production, `npm run build` instead produces a fixed set of optimised, minified files in a `dist/` folder, ready to be uploaded to a static host like Vercel.

9. React and component-based UI

React is a library for building user interfaces out of **components** — self-contained functions that return a description of what should appear on screen, written in **JSX** (a syntax extension that lets HTML-like markup appear directly inside JavaScript code). `App.jsx` is one such component; `main.jsx` takes that component and mounts it into the single `<div id="root">` element declared in `index.html` — this is why the HTML file itself is almost empty; React builds the entire visible page at run time inside that one div. This pattern is called a **Single Page Application** (SPA): the browser loads one HTML file once, and everything the user sees afterward is generated and swapped in by JavaScript, without full page reloads.

10. Tailwind CSS v4's plugin architecture

Tailwind is a **utility-first CSS framework** — instead of writing custom CSS class names and defining what they mean in a separate stylesheet, elements are styled by combining many small, single-purpose classes directly in the markup (e.g. `class="flex gap-3 rounded-lg"`). Version 4, used here, changed how Tailwind integrates into a build: earlier versions required a separate `tailwind.config.js` and a `postcss.config.js` (PostCSS is a general tool for transforming CSS with plugins, of which Tailwind is one). Version 4 instead ships as a native Vite plugin, configured directly in `vite.config.js`, with theme customisation moved into the stylesheet itself via an `@theme` block — visible in `src/index.css`, where the hot/warm/cold temperature colours used throughout the dashboard are defined once as CSS custom properties.

11. Progressive Web Apps

A **Progressive Web App** (PWA) is a website built to behave like an installed native app: it can be added to a phone's home screen, launched without a browser address bar, and given its own icon. Three ingredients make this possible:

- A `manifest.json` file (`dashboard/public/manifest.json`) — a JSON document describing the app's name, icon set, and `display: "standalone"` mode, which tells the OS to launch it full-screen with no browser chrome.
- Icon files at specific sizes (180×180 for iOS's home-screen icon, 192 and 512 for the manifest's own reference sizes).
- iOS-specific `<meta>` tags in `index.html` — notably `apple-mobile-web-app-capable`. Safari on iOS does not fully honour the web manifest standard, so this tag is what actually causes "Add to Home Screen" to open the page full-screen rather than as a bookmark that opens back inside Safari.

Two related but distinct CSS details were also added for the iPhone specifically: `viewport-fit=cover` in the viewport meta tag allows the page to draw underneath the physical notch/status bar, and `padding: env(safe-area-inset-top) ...` in the stylesheet then pulls real content back inside the safe, unobstructed area — without the second step, content would be drawn but visually cut off by the notch.

Not yet built: a full PWA also typically has a **service worker** — a background script that lets the app work offline and cache assets. That has not been added yet; the manifest and icons alone are enough for "installable, launches full-screen," which was the Phase 0 requirement. Offline support was not part of the spec.

12. The PNG file format, from first principles

The app icons needed to exist as real files for the manifest to be valid, but no image editor or library was available in this environment. Rather than leave broken image references, a PNG file was constructed directly, byte by byte, using only Python's standard library — which is possible because the PNG format, while a binary format, has a well-documented and fairly simple structure:

1. An 8-byte fixed **signature** that identifies the file as a PNG.
2. A sequence of **chunks**, each with a 4-byte length, a 4-byte type tag (like `IHDR`, `IDAT`, `IEND`), the chunk's data, and a 4-byte **CRC32** checksum of the type+data, so a corrupted chunk can be detected.
3. `IHDR` declares the image's width, height, colour depth, and colour type.
4. `IDAT` holds the actual pixel data, which PNG always stores **zlib-compressed** (the same compression algorithm used by `.zip` files) — Python's built-in `zlib` module handles this directly.
5. `IEND` marks the end of the file, with no data.

The script computed, for every pixel of every icon size, whether that pixel falls inside a simple geometric "N" letterform (two vertical strokes and a diagonal, defined by comparing each pixel's coordinates against a few line equations), coloured it accordingly, compressed the resulting raw pixel grid, and wrote out the three chunks by hand. This produced valid, real PNG files with no external dependency — a placeholder, explicitly meant to be replaced with real Nexaris branding later, but a functioning image file rather than a broken link in the meantime.

13. Client-server architecture and Backend-as-a-Service

In a traditional web project, a team writes and hosts its own backend server, which itself talks to a database it manages. **Backend-as-a-Service** (BaaS) replaces the custom backend with a managed platform that provides the common pieces — a database, user authentication, and often a realtime data feed — as a ready-made API. This project uses **Supabase**, which packages:

- A real, full **PostgreSQL** database (not a proprietary database — the same open-source relational database used by countless production systems).
- **PostgREST**, a piece of software that inspects the database's table structure and automatically generates a REST API for it — this is the layer the `supabase` Python and JavaScript packages actually talk to. No custom API server was written for this project; Supabase generated it directly from the schema.
- Authentication (email/password or other login methods), which is what the `authenticated` role referenced throughout the security policies represents.
- A realtime layer that can push database changes to connected clients over a **WebSocket** (a persistent, two-way connection, unlike a normal request/response HTTP call) — this is what will let the dashboard update live as the agent works, in a later phase, without the browser needing to repeatedly ask "has anything changed?".

14. API keys and JSON Web Tokens

Supabase issues two keys for this project, and both are, technically, a **JSON Web Token** (JWT) — a specific, standard text format for a signed piece of data. A JWT is three base64-encoded segments separated by dots:

`header.payload.signature`. The middle segment, once decoded, is a plain JSON object; for these keys it states which Supabase project the key belongs to and which **role** it grants — either `anon` or `service_role`. The signature is what stops anyone from editing that payload and forging a different role for themselves — it is produced with a secret only Supabase's own servers hold, so an altered token fails verification instantly.

The **anon key** identifies the holder as an anonymous, un-elevated client — it is meant to be public, and its safety depends entirely on server-side rules (Row Level Security, covered in section 21) actually restricting what an anonymous or logged-in client can do. The **service role key** instead identifies the holder as the database's own administrator — it bypasses every Row Level Security rule automatically. This is why the dashboard's code uses only the anon key and the agent's code uses only the service role key: the dashboard runs in a browser, in front of the public, where the key is by definition readable by anyone who opens developer tools; the agent runs on a private server under Hussein's control, where a powerful key is appropriate.

15. Relational database fundamentals

A **relational database** organises data into **tables** — grids of rows and columns, where every row represents one record (one lead, one message) and every column represents one attribute of that record (a business name, a score). Tables can reference one another, forming **relations** — a message row can point to the specific lead row it belongs to, rather than duplicating that lead's information inside every message. This project has nine tables: `accounts`, `settings`, `leads`, `messages`, `pipeline_history`, `follow_ups`, `client_history`, `notifications_log`, and `runs`.

16. SQL DDL — building the schema

SQL (Structured Query Language) is the standard language for interacting with a relational database. It divides broadly into **DDL** (Data Definition Language — statements like `CREATE TABLE` that define structure) and **DML** (Data Manipulation Language — `SELECT`, `INSERT`, `UPDATE`, `DELETE`, which read and write the data inside that structure). `02_SUPABASE_SCHEMA.sql` is pure DDL plus a small amount of seed DML at the end. Each `create table if not exists` statement lists the table's columns, each with a data type (`text`, `int`, `boolean`, `timestampz`, `jsonb`, `uuid`) and optional constraints — `not null` (the value is required), and `default` (the value used automatically if none is supplied on insert).

17. UUIDs as primary keys

Every table's `id` column is a **UUID** (Universally Unique Identifier) — a 128-bit number, conventionally written as 32 hexadecimal digits grouped like `8bda2e52-fedb-4c6f-a3b5-0300a2838ee2` (an actual id generated during the Phase 1 verification test, since deleted). It serves as the table's **primary key** — the column guaranteed to uniquely identify one row. The schema's opening line, `create extension if not exists "uuid-oss" , enables a Postgres extension providing the function that generates these values automatically (uuid_generate_v4() , referenced in every table's id default). UUIDs were chosen over a simpler auto-incrementing integer specifically because they can be generated independently, by any process, with no coordination needed and no risk of two different rows ever colliding on the same id — useful when both the agent and the dashboard might create rows concurrently.`

18. Foreign keys and referential integrity

A **foreign key** is a column in one table that holds the primary key value of a row in another table, explicitly declared with `references <table>(<column>)`. For example, `leads.account_id uuid references accounts(id)` states that every lead's `account_id` must match a real, existing row in `accounts` — the database itself enforces this, rejecting any insert that would violate it. Several foreign keys in this schema add `on delete cascade` (e.g. on `messages.lead_id`) or `on delete set null` (e.g. on `client_history.lead_id`) — these declare what should automatically happen to dependent rows if the row they reference is deleted: cascade means "delete this row too," set null means "keep this row, but clear the reference." This is **referential integrity** — the guarantee that the relationships between tables can never point at something that doesn't exist.

19. JSONB columns

Several columns (`leads.weak_points` , `leads.social_platforms`) are typed `jsonb` rather than a fixed set of ordinary columns. **JSONB** is Postgres's binary-optimised format for storing an arbitrary JSON document (an object or array) inside a single column, while still being queryable and indexable. This is a deliberate trade-off against strict relational modelling: a lead might have three weak points or ten, and they don't need their own separate table with a foreign key back to the lead — a JSON array in one column is simpler here, at the cost of that data being less directly queryable by SQL than a normal column would be. This is sometimes called a controlled use of a **document-style** pattern inside an otherwise relational schema.

20. Indexes

A plain table lookup by a non-key column normally requires scanning every row. An **index** is a separate, ordered data structure the database maintains alongside a table specifically to make lookups on one column fast — conceptually similar to a book's index letting you jump straight to a page instead of reading cover to cover. The schema creates eight indexes, e.g. `create index idx_leads_status on leads(status)` , chosen on the columns this project will filter by constantly (a lead's lifecycle status, its temperature, which platform it came from) — these are the queries the dashboard and agent will run most often, so they are the ones worth the (small) extra storage and write cost an index carries.

21. Row Level Security, in depth

Creating a table in Postgres does not, by itself, restrict who can read or write it — access control is a separate, explicit layer called **Row Level Security** (RLS). Turning it on for a table (`alter table <t> enable row level security`) flips the table's default from "accessible" to "nothing is accessible until a policy says otherwise." A **policy** then grants a specific carve-out:

```
create policy "authenticated full access" on accounts
  for all to authenticated using (true) with check (true);
```

Read this as: for every operation (`for all` — select, insert, update, delete), for requests authenticated as a logged-in user (`to authenticated`), allow it unconditionally. The `using (true)` clause governs which existing rows may be read/updated/deleted (here, all of them); `with check (true)` governs which new or edited rows are allowed to be written (here, anything). Because there are exactly two trusted users on this system, both clauses are simply `true` — a more selective system might instead write `using (owner_id = auth.uid())` , restricting each user to only their own rows.

Crucially, RLS is evaluated per request based on which *role* the connection is using — and the service role key (section 14) is defined by Supabase to bypass RLS entirely, by design, regardless of any policy written. This is why the same nine `enable row level security` statements that lock the dashboard down completely do not restrict the agent at all: the agent authenticates as `service_role` , never as `authenticated` .

22. The Repository Pattern

`agent/db/repositories.py` implements a well-known software design pattern called the **Repository Pattern**: rather than letting every part of a program construct its own database queries wherever it needs data, all data access is funnelled through one layer of small, purpose-named functions (`get_settings()` , `insert_lead(...)`). The rest of the codebase depends only on this layer's function signatures, never on how the data is actually stored or fetched underneath. The practical benefit shows up the moment the schema changes: a renamed column, a new table, or even a switch to a different database entirely would only require editing this one file — every caller elsewhere in the codebase is unaffected because it never referenced a table or column name directly.

23. REST over Postgres — how the client actually talks to the database

The Python code never opens a raw database connection or writes SQL by hand during normal operation (Phase 1's one-time schema migration, discussed in section 25, was the sole exception). Instead,

`get_client().table("leads").select("*").eq("status", "discovered").execute()` builds an HTTP request under the hood, sent to the PostgREST API that Supabase generates automatically from the schema (section 13). This is a **REST** (Representational State Transfer) style API: resources (tables) are addressed by URL, and standard HTTP verbs (GET, POST, PATCH, DELETE) map onto read/create/update/delete operations. The Python `supabase` package is a thin, convenient wrapper that constructs these HTTP requests and parses their JSON responses, rather than an implementation of the database protocol itself.

24. Testing: smoke tests and round-trip verification

`smoke_test.py` is what is generally called a **smoke test** — a fast, broad check that the overall system is basically alive and functioning, run before investing time in anything more detailed (the name comes from hardware testing: power on a device and check that smoke doesn't come out, before testing individual features). It performs what is specifically called a **round-trip test** on write access: insert a record, read it back, and assert the two match exactly, which proves not just that a write "succeeded" in isolation but that what comes back out is genuinely what went in — then deletes that record so the test has no lasting side effect on the real data, a property called being **idempotent**-safe to re-run.

25. Networking — DNS, IPv4 vs IPv6, connection pooling

Connecting directly to the database (rather than through the PostgREST API) required a raw Postgres connection string and hit a real networking issue worth explaining properly.

DNS resolution

A hostname like `db.xtesxgezongzpdgppobm.supabase.co` is not itself an address a computer can connect to — it must first be translated into a numeric **IP address** via the **Domain Name System** (DNS), the internet's distributed lookup service mapping names to addresses. The error encountered, `could not translate host name ... to address`, is a DNS resolution failure specifically — the name could not be turned into any usable address at all on this network.

IPv4 versus IPv6

There are two addressing schemes in use across the internet: **IPv4** (the older, more universally supported scheme, addresses like `192.0.2.1`, with a limited total address space that has effectively run out) and **IPv6** (the newer scheme designed to have effectively unlimited addresses, written like `2001:db8::1`). Supabase's direct-connection hostname resolves only to an IPv6 address. Many home and office networks, including the one this project was built on, either have no IPv6 connectivity configured or route it unreliably — so a hostname that only exists in IPv6 space is entirely unreachable from such a network, which is exactly the failure observed.

Connection pooling

The fix was switching to Supabase's **session pooler** endpoint instead — a separate hostname that does resolve over IPv4. A **connection pooler** is a piece of infrastructure sitting between a client and the actual database, holding open a smaller, reused set of real database connections and multiplexing many client requests across them, rather than every client opening its own direct connection to Postgres (each of which carries real memory and CPU overhead on the database server). Supabase offers pooling in two modes — session pooler (one pooled connection held for the lifetime of a client session) and transaction pooler (a connection is only held for the duration of a single transaction, then immediately returned to the pool) — the session mode was used here since it is what Supabase's own UI recommended for the equivalent scripted, persistent-connection use case.

26. Credential hygiene and the principle of least privilege

Several practices in this project follow the general security principle of **least privilege**: every component should hold only the access it actually needs, and no more. Concretely:

- The dashboard, which runs in an untrusted context (any visitor's browser), was given only the anon key, restricted further by Row Level Security.
- The agent, which runs in a trusted context Hussein controls, was given the powerful service role key — but only in a file (`agent/.env`) that is never transmitted anywhere and is explicitly excluded from version control.
- The one-time raw database password used to run the initial migration was used inside a single temporary script and then deleted immediately afterward, rather than being written into any permanent project file — it was needed for one administrative action, not for the system's ongoing operation, so it was not given ongoing storage.

The broader habit this reflects: before adding a secret anywhere, ask what the minimum access required for the task actually is, and grant exactly that — not the most convenient or most powerful credential available.

27. Every file created, by category

Category	Files
Version control config	<code>.gitignore</code>
Environment/config	<code>.env.example</code> , <code>agent/config.py</code> , <code>agent/.env</code> , <code>dashboard/.env</code> (both untracked)
Python packaging	<code>agent/requirements.txt</code> , 9 × <code>__init__.py</code>
Database connectivity (Python)	<code>agent/db/client.py</code> , <code>agent/db/repositories.py</code> , <code>agent/db/smoke_test.py</code>
SQL — schema and security	<code>02_SUPABASE_SCHEMA.sql</code> , <code>database/policies.sql</code> , <code>database/schema.sql</code>
Agent entry point	<code>agent/main.py</code>
Agent stub modules (23)	<code>core/</code> , <code>discovery/</code> , <code>analysis/</code> , <code>messaging/</code> , <code>sending/</code> , <code>crm/</code> , <code>notifications/</code>
JavaScript packaging	<code>dashboard/package.json</code> , <code>dashboard/vite.config.js</code>
Frontend application	<code>dashboard/index.html</code> , <code>src/main.jsx</code> , <code>src/App.jsx</code> , <code>src/index.css</code>
Frontend data layer	<code>dashboard/src/lib/supabase.js</code>
PWA assets	<code>dashboard/public/manifest.json</code> , 4 generated icon files
Tooling	<code>tools/make_pdf.py</code>
Project documentation	<code>README.md</code> , <code>PROGRESS.md</code> , <code>REQUIREMENTS_COVERAGE.md</code> , this document, and the per-phase PDFs in <code>docs/</code>

28. Glossary of every term used

API (Application Programming Interface)

A defined way for one program to request data or actions from another.

BaaS (Backend-as-a-Service)

A hosted platform providing ready-made backend building blocks (database, auth, realtime) instead of a custom-built server.

Build tool

Software that converts source code into the plain form a runtime (e.g. a browser) can execute.

Commit

A saved snapshot of a project's files in Git, with a message describing the change.

Component (React)

A self-contained function describing one piece of a user interface.

Connection pooler

Infrastructure that reuses a small set of database connections across many clients instead of one connection per client.

CRC32

A checksum algorithm used to detect corrupted data, used inside the PNG format.

DDL / DML

SQL statements that define structure (DDL: CREATE TABLE) versus statements that read/write data (DML: SELECT, INSERT).

DNS (Domain Name System)

The internet service that translates hostnames into IP addresses.

Environment variable

A named configuration value supplied to a program from outside its source code.

Foreign key

A column whose value must match a primary key in another table, enforcing that a relationship points at something real.

Git

A version control system that tracks changes to files as a chain of commits.

Index (database)

An extra data structure that speeds up lookups on a specific column, at the cost of some storage and write overhead.

IPv4 / IPv6

Two internet addressing schemes; IPv4 is older and more universally reachable, IPv6 has a far larger address space but incomplete network support.

Idempotent

An operation that can be safely repeated without changing the outcome beyond the first run.

JSONB

PostgreSQL's binary-optimised column type for storing a JSON document.

JWT (JSON Web Token)

A signed, three-part text token (header, payload, signature) commonly used to carry identity/role claims.

Least privilege

The security principle of granting only the minimum access a component actually needs.

Node.js

A runtime that executes JavaScript outside a browser.

npm

Node's package manager and public package registry.

Primary key

The column (or columns) guaranteed to uniquely identify one row in a table.

PostgREST

Software that auto-generates a REST API directly from a Postgres database's schema.

PWA (Progressive Web App)

A website built to be installable and to behave like a native app.

Referential integrity

The database-enforced guarantee that foreign key relationships always point at an existing row.

Repository pattern

A design pattern centralising all data access behind one set of named functions.

REST

An API style where resources are addressed by URL and manipulated via standard HTTP verbs.

RLS (Row Level Security)

A PostgreSQL feature restricting which rows a given database role may read or write, via explicit policies.

Round-trip test

A test that writes data and reads it back to confirm the two match exactly.

Separation of concerns

A design principle: each part of a system should have exactly one responsibility.

Service worker

A background browser script enabling offline behaviour in a web app; not used in this phase.

Smoke test

A fast, broad check that a system is basically functioning, run before deeper testing.

SPA (Single Page Application)

A web app that loads one HTML page once and updates the view via JavaScript afterward, without full reloads.

SQL

Structured Query Language — the standard language for interacting with a relational database.

Staging area (Git)

The intermediate step where changes are marked for inclusion in the next commit.

UUID

A 128-bit identifier that can be generated independently, with negligible chance of collision, without central coordination.

Virtual environment (venv)

An isolated, project-scoped copy of a Python interpreter and its installed packages.

WebSocket

A persistent, two-way network connection, as opposed to one-off request/response HTTP calls.

Zlib

A general-purpose compression algorithm, used inside the PNG format for pixel data.

Technical deep dive covering Phases 0 and 1 · Generated 24 July 2026

Source: `docs/TECHNICAL_DEEP_DIVE_PHASE_0_1.html` · Rendered by `tools/make_pdf.py`